Локальная область видимости

Как мы уже знаем из предыдущих уроков, при выполнении процессором **стейтмента** `int x;` создается переменная. Возникает вопрос: «Когда эта переменная уничтожается?».

Область видимости переменной определяет, кто может видеть и использовать переменную во время её существования. И параметры функции, и переменные, которые объявлены внутри функции, имеют **локальную область видимости**. Другими словами, эти параметры и переменные используются только внутри функции, в которой они объявлены. Локальные переменные создаются в точке объявления и уничтожаются, когда выходят из области видимости.

Рассмотрим следующую программу:

```
#include <iostream>

int add(int a, int b) // здесь создаются переменные a и b
{
   // a и b можно видеть/использовать только внутри этой функции
   return a + b;
} // здесь a и b выходят из области видимости и уничтожаются

int main()
{
   int x = 7; // здесь создается и инициализируется переменная x
   int y = 8; // здесь создается и инициализируется переменная y
   // x и y можно использовать только внутри функции main()
   std::cout << add(x, y) << std::endl; // вызов функции add() с a = x и b = y
   return 0;
} // здесь x и y выходят из области видимости и уничтожаются
```

Параметры `a` и `b` функции add() создаются при вызове этой функции, используются только внутри нее и уничтожаются по завершении выполнения этой функции.

Переменные `x` и `y` функции main() можно использовать только внутри main() и они также уничтожаются по завершении выполнения функции main().

Для лучшего понимания давайте детально разберем ход выполнения этой программы:

- выполнение начинается с функции main();
- создается переменная `x` в функции main() и ей присваивается значение `7`;
- создается переменная `y` в функции main() и ей присваивается значение `8`;
- вызывается функция add() с **параметрами** `7` и `8`;
- создается переменная `a` в функции add() и ей присваивается значение `7`;
- создается переменная `b` в функции add() и ей присваивается значение `8`;
- выполняется операция сложения чисел `7` и `8`, результатом является значение `15`;
- функция add() возвращает значение `15` обратно в caller (в функцию main());
- переменные `a` и `b` функции add() уничтожаются;
- функция main() выводит значение `15` на экран;
- функция main() возвращает `0` в операционную систему;
- переменные `x` и `y` функции main() уничтожаются.

Всё!

Обратите внимание, если бы функция add() вызывалась дважды, параметры `a` и `b` создавались и уничтожались бы также дважды. В программе с большим количеством функций, переменные создаются и уничтожаются часто.

Локальная область видимости предотвращает возникновение конфликтов имен

Из примера, приведенного выше, понятно, что переменные `x` и `y` отличаются от переменных `a` и `b`.

Теперь давайте рассмотрим следующую программу:

```
#include <iostream>

int add(int a, int b) // здесь создаются переменные a и b функции add()
{
   return a + b;
} // здесь a и b функции add() выходят из области видимости и уничтожаются

int main()
{
   int a = 7; // здесь создается переменная a функции main()
   int b = 8; // здесь создается переменная b функции main()
   std::cout << add(a, b) << std::endl; // значения переменных a и b функции main() копируются в переменные a и b функции add(
   return 0;
} // здесь a и b функции main() выходят из области видимости и уничтожаются
```

Здесь мы изменили имена переменных `x` и `y` функции main() на `a` и `b`. Программа по-прежнему работает корректно, несмотря на то, что функция add() также имеет переменные `a` и `b`. Почему это не вызывает конфликта имен? Дело в том, что `a` и `b`, принадлежащие функции main(), являются локальными переменными, функция add() не может их видеть, точно так же, как функция main() не может видеть переменные `a` и `b`, принадлежащие функции add(). Ни add(), ни main() не знают, что они имеют переменные с одинаковыми именами!

Это значительно снижает возможность возникновения конфликта имен. Любая функция не должна знать или заботиться о том, какие переменные находятся в другой функции. Это также предотвращает возникновение ситуаций, когда одни функции могут непреднамеренно (или намеренно) изменять значения переменных других функций.

Правило: Имена, которые используются внутри функции (включая параметры), доступны/видны только внутри этой же функции.

Тест

Каким будет результат выполнения следующей программы?

```
#include <iostream>

void doMath(int a)
{
    int b = 5;
    std::cout << "doMath: a = " << a << " and b = " << b << std::endl;
    a = 4;
    std::cout << "doMath: a = " << a << " and b = " << b << std::endl;
}

int main()
{
    int a = 6;
    int b = 7;
    std::cout << "main: a = " << a << " and b = " << b << std::endl;
    doMath(a);
    std::cout << "main: a = " << a << " and b = " << b << std::endl;
    return 0;
}
```

Ответ

Результат выполнения программы:

```
main: a = 6 and b = 7
doMath: a = 6 and b = 5
doMath: a = 4 and b = 5
main: a = 6 and b = 7
```

Вот ход выполнения этой программы:

- выполнение начинается с функции main();
- создается переменная `a` в функции main(), ей присваивается значение `6`;
- создается переменная `b` в функции main(), ей присваивается значение `7`;
- **cout** выводит `main: a = 6 and b = 7`;
- вызывается doMath() с аргументом `6`;
- создается переменная `a` в функции doMath(), ей присваивается значение `6`;
- выполняется инициализация переменной `b` функции doMath() значением `5`;
- cout выводит `doMath: a = 6 and b = 5`;
- переменной `a` функции doMath() присваивается значение `4`;
- cout выводит `doMath: a = 4 and b = 5`;
- переменные `a` и `b` функции doMath() уничтожаются;
- cout выводит `main: a = 6 and b = 7`;
- функция main() возвращает `0` в операционную систему;
- переменные `a` и `b` функции main() уничтожаются.

Обратите внимание, даже когда мы присвоили значения переменным `a` и `b` внутри функции doMath(), на переменные внутри функции main() это никак не повлияло.